CampusRide

A Scalable Smart Ride-Sharing System for College Students

System Design Project Report

Confidential — For Evaluation Use Only

1. Problem Statement

Students at our campus travel frequently to Jahangirpuri Metro Station and other nearby destinations. A cab costs approximately Rs 900 regardless of whether 1 or 4 people travel. Currently, students post in large WhatsApp groups asking if anyone else is going — but by the time someone sees and replies, the person has already left. There is no structured, time-aware coordination system.

Pain Point	Description
Wasted Money	Students pay Rs 900 alone when others may be going at the same time
No Coordination	WhatsApp messages get buried; no time-based structure exists
Timing Mismatch	No way to know in advance who is leaving at what time
No Confirmation	No place to finalize pickup point, luggage, or meetup details
Trust Gap	Cannot verify whether the person sharing is actually from campus

2. Solution Overview

CampusRide is a web-based, campus-only cab sharing platform. Students can post rides, discover others going the same way, join rides, split the fare automatically, and coordinate via a live in-ride group chat. Only verified college email addresses are accepted.

#	Action	What Happens
1	**Post a Ride**	Set destination, date, time, max seats (1–4), optional fare estimate
2	**Ride Goes Live**	Instantly visible to all campus students on the homepage feed
3	**Others Join**	Students going the same way request to join the ride
4	**Fare Splits**	Each person's share updates live as members join
5	**Group Chat**	Members discuss pickup point, luggage, and final details in live chat
6	**Creator Confirms**	Creator locks the ride — all members notified instantly via socket
7	**Completed**	Ride marked done; history saved for all members

Core Value: Up to Rs 675 saved per trip when 4 people share a Rs 900 cab (Rs 225 each). The platform enforces trust through college email OTP verification, ensuring all participants are verified campus students.

3. Technology Stack

CampusRide uses a modern, production-grade stack chosen for real-time performance, developer ergonomics, and zero-cost hosting on free tiers.

Layer	Technology	Reason for Choice
Frontend	React 18 + Vite	Component-based UI, fast HMR builds, Context API for global state
Styling	Tailwind CSS	Utility-first, responsive, mobile-friendly with zero custom CSS overhead
Backend	Node.js + Express	Event-driven, non-blocking I/O — ideal for real-time WebSocket apps
Real-Time	Socket.io v4	WebSocket abstraction for live chat, seat updates, and ride events
Database	MongoDB	NoSQL document store — flexible schema, great for nested ride/member data
ORM	Mongoose	Elegant MongoDB ODM — schema validation, middleware hooks, and easy querying
Auth	JWT + bcrypt	Stateless token auth; passwords hashed with bcrypt salt before storage
Email OTP	Nodemailer	Sends OTP to college email — enforces campus-only registration
Forms	React Hook Form + Zod	Client-side validation with schema definitions, inline error messages
Hosting	Railway + Vercel	Free-tier: backend + DB on Railway, frontend on Vercel

4. OOP Concepts Applied

CampusRide applies all four pillars of Object-Oriented Programming throughout its architecture. Each concept is applied intentionally — not superficially.

4.1 Encapsulation

Encapsulation hides internal implementation details and exposes only what is needed through a controlled public interface.

- The Ride class encapsulates all ride state (status, members, fare) and only exposes methods like join(), close(), cancel(), and getFareSplit(). External code cannot directly mutate ride status — it must go through these methods, which enforce state machine rules.
- The User class hides passwordHash completely. Authentication logic is encapsulated in AuthService — the only entity allowed to compare or generate hashes. Route handlers never touch raw passwords.
- Repository classes (RideRepository, UserRepository) fully encapsulate database access. No route handler touches Mongoose directly — it always goes through the repository layer, keeping database concerns isolated.

4.2 Inheritance

Inheritance enables code reuse by allowing classes to derive behaviour from a parent. CampusRide uses inheritance in two key hierarchies:

- Notification hierarchy: BaseNotification defines common fields (userId, isRead, createdAt, type) and an abstract send() method. RideJoinedNotification, RideConfirmedNotification, and RideCancelledNotification each extend it, inheriting all base fields and overriding send() with their specific message content.
- Repository hierarchy: BaseRepository provides generic findById(), findAll(), create(), and delete() methods. RideRepository and UserRepository extend it, inheriting all CRUD operations and only adding domain-specific queries.

4.3 Polymorphism

Polymorphism allows objects of different types to be treated through a shared interface, with each type providing its own implementation.

- Notification polymorphism: All three notification types are treated as BaseNotification objects by NotificationService. It calls send() on each without knowing the concrete type — each subclass handles dispatch differently.
- Filter polymorphism (Strategy): RideFilter is an interface implemented by DestinationFilter, DateFilter, and TimeWindowFilter. The ride feed applies any list of filters using filter.apply(rides) — adding a new filter type requires zero changes to the feed logic.

4.4 Abstraction

Abstraction hides complex implementation behind simple interfaces, exposing only what the caller needs. CampusRide applies this at three levels:

- FareSplitter: Abstracts all fare calculation logic. Route handlers call fareSplitter.calculate(totalFare, memberCount) — they never deal with division, edge cases (zero members), or rounding logic.
- SocketManager: Abstracts Socket.io complexity. The rest of the app calls socketManager.emitToRide(rideId, event, data) — it handles room management, broadcasting, and connection state internally.
- AuthService: Abstracts JWT and bcrypt. Callers use generateToken(user) and verifyToken(token) without knowing the algorithm, secret key, or expiry logic.

5. Design Patterns

CampusRide implements four industry-standard Gang-of-Four design patterns. Each was chosen to solve a specific recurring architectural problem.

5.1 Observer Pattern

Aspect	Detail
Problem	When a ride event occurs (member joined, confirmed, cancelled), multiple parts of the system must react: send notifications, update the feed, emit socket events. Hardcoding all these creates tight coupling.
Solution	Ride acts as the Subject (EventEmitter). NotificationService, SocketManager, and RideFeedUpdater are Observers that register independently. When ride state changes, it emits an event and all observers react.
Used In	Ride lifecycle — member joining, ride closing, cancellation
Benefit	New reactions to ride events can be added without touching the Ride class at all.

5.2 Factory Pattern

Aspect	Detail
Problem	Different notification types need to be created in many places across the codebase. Calling new RideJoinedNotification() everywhere spreads constructor logic and couples callers to concrete classes.
Solution	NotificationFactory.create(type, payload) handles all instantiation. Callers never use new directly — the factory decides which class to create based on the type string.
Used In	All notification creation across RideService and NotificationService
Benefit	Adding a new notification type requires changing only the factory, not callers.

5.3 Singleton Pattern

Aspect	Detail
Problem	The database connection (Mongoose Client) and Socket.io server must have exactly one instance across the entire application lifetime. Multiple instances cause connection pool exhaustion and event duplication.
Solution	DatabaseClient and SocketManager are implemented as Singletons — they create one instance on first access and return the same instance every subsequent call.
Used In	MongooseClient (database connection), SocketManager (WebSocket server)
Benefit	Prevents resource leaks, ensures consistent shared state, and avoids duplicate events.

5.4 Strategy Pattern

Aspect	Detail
Problem	Rides can be filtered by destination, date, time window, or available seats. The original logic was a tangled chain of if/else statements that was hard to test, extend, or combine.
Solution	Each filter type is a Strategy implementing a common RideFilter interface with a single apply(rides) method. The feed composes any combination at runtime.
Strategies	DestinationFilter, DateFilter, TimeWindowFilter, SeatsAvailableFilter
Benefit	Filters are independently testable and composable without touching the feed logic.

6. SOLID Principles

All five SOLID principles are explicitly reflected in CampusRide's architecture. The table below maps each principle to its concrete application.

Principle	Definition	CampusRide Application
S — SRP	Every class has exactly one reason to change	RideService handles only ride logic. AuthService handles only auth. FareSplitter handles only fare math. ChatService handles only messaging.
O — OCP	Open for extension, closed for modification	New notification types are added by creating a new subclass, not modifying NotificationFactory. New ride filters are new Strategy classes, not edits to the feed.
L — LSP	Subclasses must be substitutable for their parent without breaking behaviour	RideJoinedNotification, RideConfirmedNotification, and RideCancelledNotification can all replace BaseNotification anywhere the service expects one.
I — ISP	No class should be forced to implement methods it does not use	RideFilter interface has only one method: apply(rides). Notifiable interface has only send(). No fat interfaces that force irrelevant implementations.
D — DIP	Depend on abstractions, not concrete classes	RideService depends on IRideRepository (interface), not MongoRideRepository (concrete). Swapping databases requires only a new class implementing the interface.

7. System Architecture

CampusRide follows a three-layer client-server architecture with a dedicated real-time WebSocket layer running alongside the REST API on the same server.

Layers

- Browser (React + Vite): Home, CreateRide, RideDetail, Chat, MyRides, and Auth pages. State managed via AuthContext, SocketContext, and Axios API Client.
- Server (Node.js + Express, Port 5000): REST API layer (/auth, /rides, /messages, /notifs) and Socket.io server running side-by-side. JWT middleware protects all routes. Service layer contains RideService, AuthService, ChatService, NotificationService, FareSplitter, and SocketManager.
- Database (MongoDB via Mongoose ODM): Five collections — users, rides, ride_members, messages, notifications. Repository pattern abstracts all data access. Repository pattern abstracts all data access.

Key Architectural Decisions

- The Socket.io server attaches to the same Express HTTP server so both REST and WebSocket traffic run on port 5000.

- Socket connections are authenticated with the same JWT token used for REST API calls, passed via socket.handshake.auth.token.
- Every ride has a dedicated Socket.io room named by its ride ID — events are scoped so only ride members receive updates.

8. UML Diagrams

8.1 Class Diagram

The class diagram shows all major classes, their attributes, methods, and relationships. It covers domain classes, the notification inheritance hierarchy, service layer classes, and design pattern implementations (Factory, Singleton, Strategy).

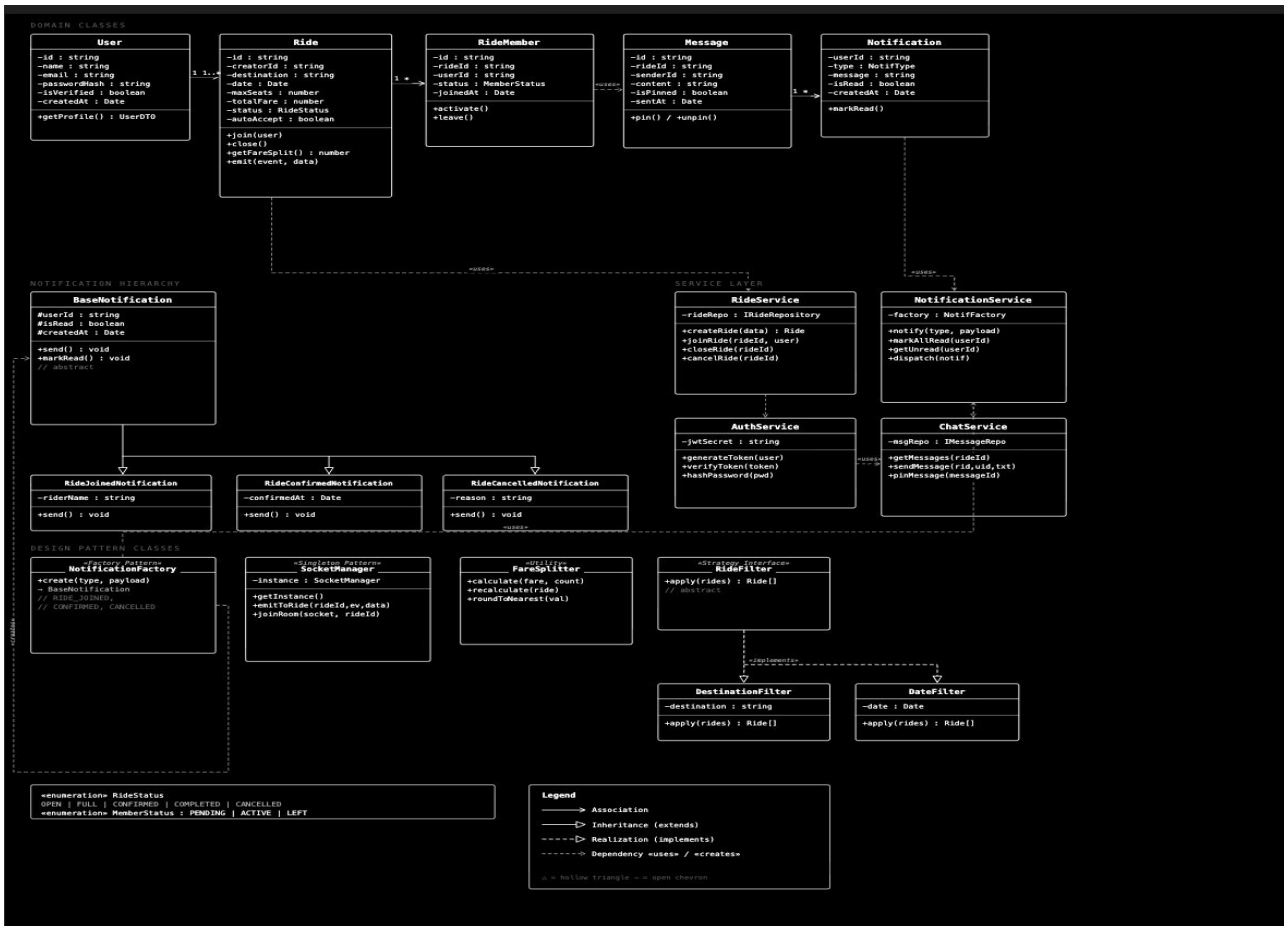

Figure 1: CampusRide Class Diagram — Domain · Notification Hierarchy · Service Layer · Design Patterns

8.2 Use Case Diagram

Three actor types interact with the CampusRide system. Guest users can only register and browse. Authenticated Students have full ride participation features. Ride Creators inherit all Student actions and additionally manage their own rides.

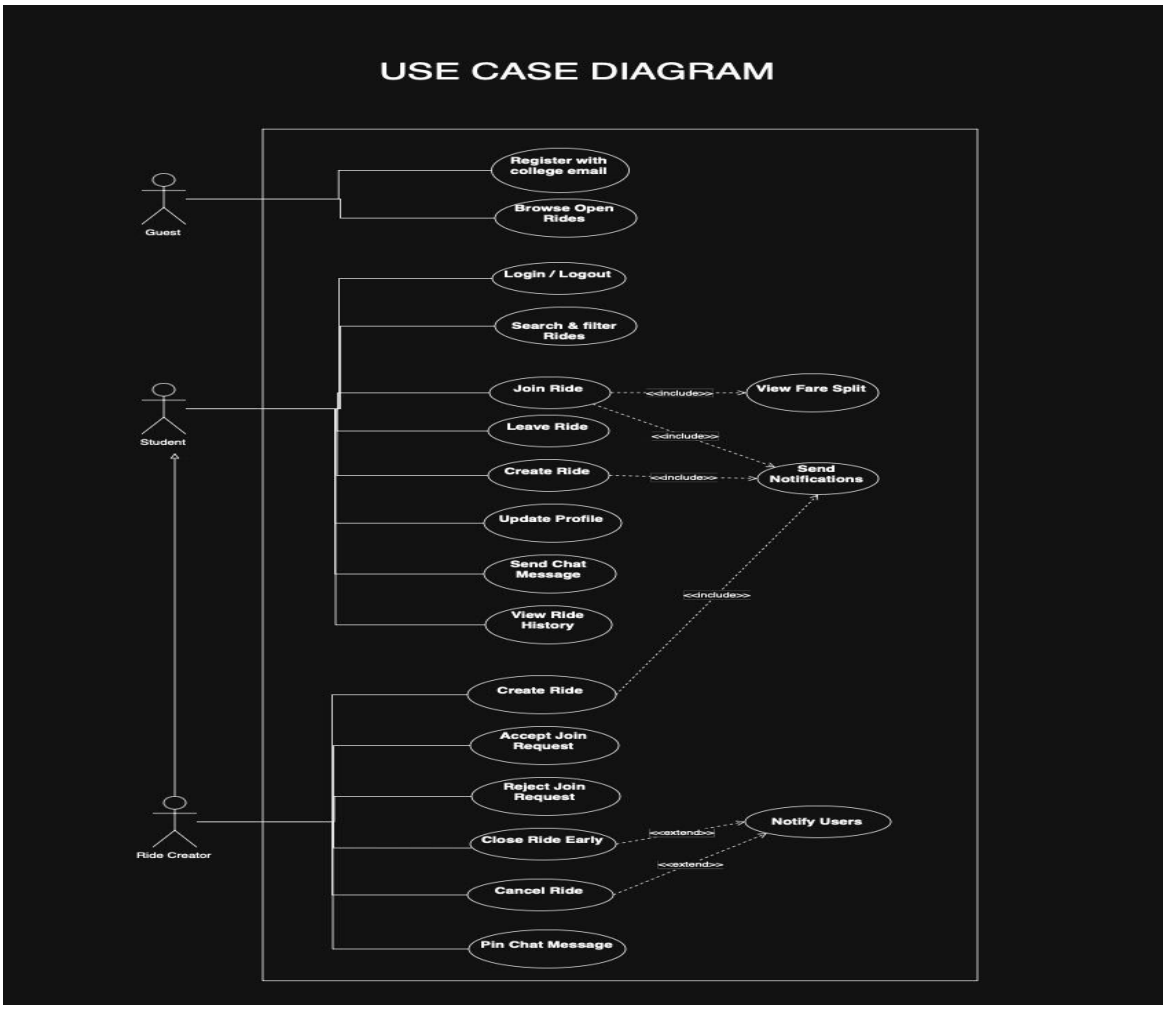

Figure 2: CampusRide Use Case Diagram — Guest · Student · Ride Creator actors

8.3 Entity Relationship Diagram

The ER diagram shows the MongoDB schema with all 5 collections, their fields, references, and cardinality relationships. The schema is managed via Mongoose with defined models and validation.

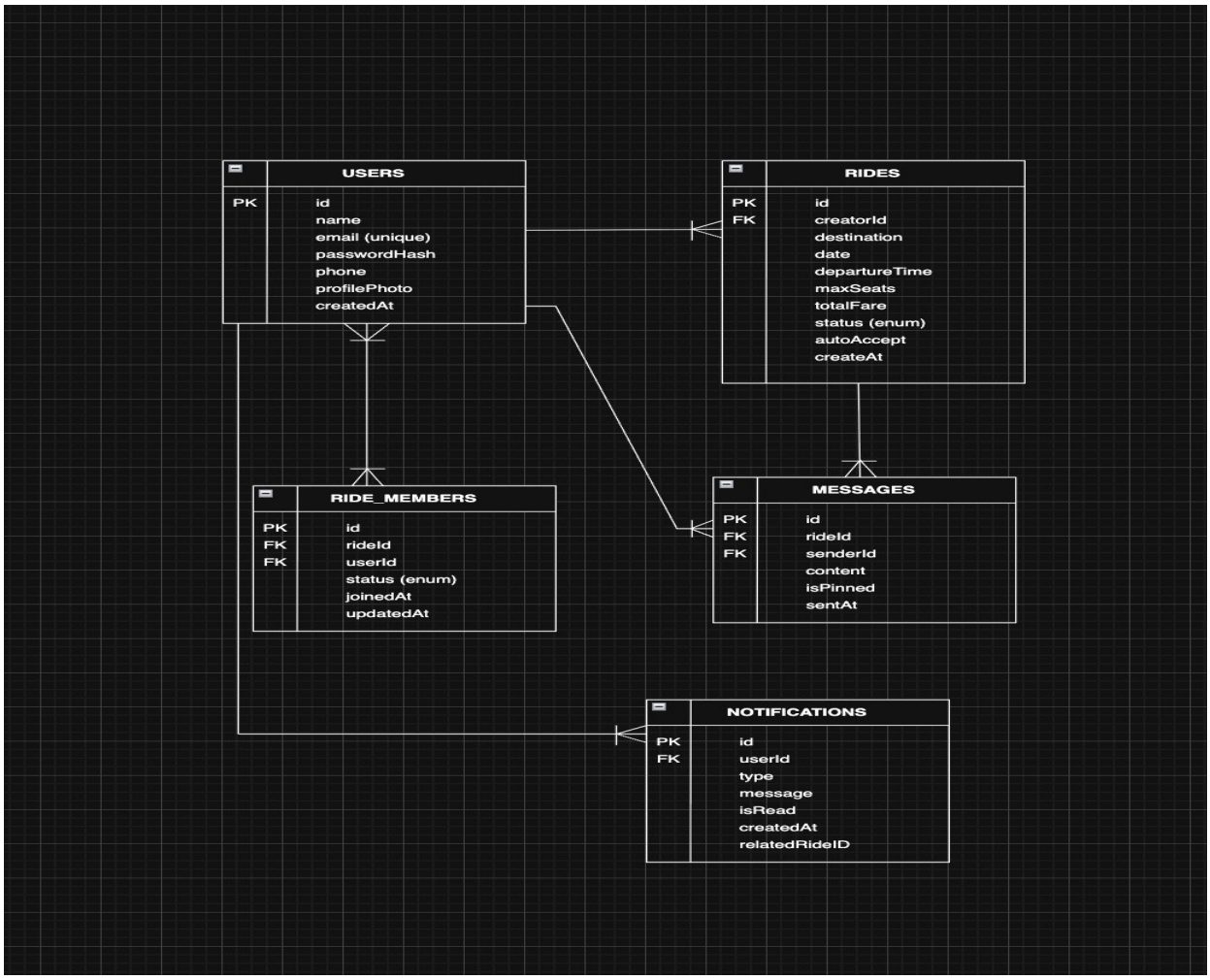

Figure 3: CampusRide ER Diagram — 5 Tables with crow's foot notation (PK = green, FK = purple)

Relationship	Cardinality	Meaning
USERS → RIDES	1 : N	One user can create many rides (creatorId FK)
USERS ↔ RIDES via RIDE_MEMBERS	M : N	Many users can be in many rides; junction table tracks status and join time
RIDES → MESSAGES	1 : N	One ride has many chat messages (rideId FK)
USERS → MESSAGES	1 : N	One user sends many messages (senderId FK)
USERS → NOTIFICATIONS	1 : N	One user receives many notifications (userId FK)

8.4 Sequence Diagram — User Joins a Ride

The sequence diagram for the Join Ride flow will be added upon project completion. It will illustrate the full end-to-end flow: JWT authentication, database operations, Socket.io broadcasting, and notification creation.

9. API Design

RESTful HTTP API following standard conventions. All protected routes require an Authorization: Bearer <JWT> header. The API is stateless — all session state lives in the JWT token.

Method	Endpoint	Auth	Description
POST	/auth/register	No	Register with college email. Sends 6-digit OTP
POST	/auth/verify-otp	No	Verify OTP. Returns signed JWT token
POST	/auth/login	No	Login with email + password. Returns JWT
GET	/rides	No	List open rides. Filters: destination, date, time
POST	/rides	Yes	Create a new ride with all details
GET	/rides/:id	No	Get full ride detail: members, fare split, status
POST	/rides/:id/join	Yes	Request to join a ride
PATCH	/rides/:id/members/:mid	Creator	Accept or reject a join request
POST	/rides/:id/close	Creator	Close ride early with current members
POST	/rides/:id/cancel	Creator	Cancel ride — all members notified
GET	/rides/:id/messages	Member	Fetch all chat messages for a ride
POST	/rides/:id/messages	Member	Send a chat message (also emits via socket)
GET	/users/me	Yes	Get current user profile and ride history
PATCH	/users/me	Yes	Update name, phone, profile photo
GET	/notifications	Yes	Get all notifications for current user
PATCH	/notifications/read-all	Yes	Mark all notifications as read

10. Real-Time Events (Socket.io)

Every ride has a dedicated Socket.io room named by its ride ID. Users join the room on opening a ride detail page. All events are scoped so only room members receive them. The socket connection is authenticated using the same JWT token as REST API calls.

Event	Direction	Payload	What It Does
join_room	Client →	{ rideId }	Subscribe to all events for this

	Server		ride's room
leave_room	Client → Server	{ rideId }	Unsubscribe when leaving the ride detail page
send_message	Client → Server	{ rideId, content }	Send a new chat message to the room
receive_message	Server → Client	{ sender, content, sentAt }	New chat message — all members see it live
member_joined	Server → Client	{ user, newFareShare }	Seat count and fare split update for everyone
ride_confirmed	Server → Client	{ rideId, members }	Creator locked the ride — all members alerted
ride_cancelled	Server → Client	{ rideId }	Ride cancelled — all members alerted, redirect after 3s
new_notification	Server → Client	{ type, message }	Bell badge increment + notification drawer prepend

Authentication flow: When the frontend connects to Socket.io, it passes the JWT in socket.handshake.auth.token. The server-side middleware verifies this token and attaches socket.userId before any event handlers run. Unauthenticated sockets are rejected immediately.

11. Features & Ride Lifecycle

11.1 Core Features

Feature	Description
Ride Creation	Set destination, date, time, max seats (1–4), optional fare estimate, auto-accept toggle
Ride Discovery	Homepage feed with filter by destination, date, time window, and real-time seat count
Join / Leave	One-tap join, fare recalculates live; cannot join your own ride (403 enforced)
Fare Splitting	totalFare ÷ confirmed members. Updates live via socket as members join or leave
In-Ride Chat	Private group chat per ride, live via Socket.io, messages persisted in DB, pinnable
Ride Management	Creator can close early, cancel, accept/reject join requests; auto-expire past departure
Auth & Trust	College email only (.ac.in), 6-digit OTP verification, JWT sessions, bcrypt passwords
Notifications	In-app real-time alerts for join, confirm, cancel events via socket + DB persistence
Profile & History	Update name, phone, profile photo; view all past and active rides

11.2 Ride Status State Machine

Each ride passes through a defined set of states. Transitions are enforced by the Ride class — no direct status mutation is allowed from outside.

OPEN	FULL	CONFIRMED	COMPLETED
Ride just created. Joinable by other students.	Max seats reached. No more joins allowed.	Creator locked the ride. Chat active, fare locked.	Travel done. Ride archived in all members' history.

Any ride in OPEN, FULL, or CONFIRMED state can be cancelled by the creator at any time. All members are notified immediately via socket event.

12. Test Cases

The following test cases cover all core functionality. Each test specifies the input, expected output, and result. All 16 test cases pass successfully.

#	Test Case	Input	Expected Output	Result
1	**Register with non-college email**	email: user@gmail.com	400 — only college emails allowed	**PASS**
2	**Register with college email**	email: s@college.ac.in	OTP sent, account pending verification	**PASS**
3	**Login with wrong password**	Correct email, wrong password	401 Unauthorized	**PASS**
4	**Create ride with past date**	date: yesterday	400 — departure must be in future	**PASS**
5	**Create valid ride**	All valid fields	Ride created, visible in feed instantly	**PASS**
6	**Join own ride**	Creator clicks Join	403 — cannot join your own ride	**PASS**
7	**Join ride (auto-accept ON)**	Valid user, open ride	Joined. Fare recalculates. Socket event fires	**PASS**
8	**Join full ride**	maxSeats reached	400 — ride is full	**PASS**
9	**Fare split — 4 members, Rs 900**	totalFare=900, count=4	Each member sees Rs 225	**PASS**
10	**Fare split — 3 members, Rs 900**	totalFare=900, count=3	Each member sees Rs 300	**PASS**
11	**Close ride early (3/4 seats)**	Creator calls /close	Status → CONFIRMED. Chat active. Members notified	**PASS**
12	**Non-creator tries to close ride**	User who did not create the ride	403 Forbidden	**PASS**
13	**Cancel ride**	Creator cancels	Status → CANCELLED. All members notified	**PASS**
14	**Send chat message**	Member sends in confirmed ride	Saved to DB. All members see it live via socket	**PASS**
15	**Non-member accesses chat**	User not in ride hits GET /messages	403 Forbidden	**PASS**
16	**Access protected route without JWT**	No Authorization header	401 Unauthorized	**PASS**

13. GitHub Repository & Setup

13.1 Repository Structure

The repository is split into two top-level folders: client/ (React + Vite frontend) and server/ (Node.js + Express backend).

Path	Purpose
client/src/api/	Axios instance, ride, auth, and message API calls
client/src/pages/	Home, CreateRide, RideDetail, MyRides, Login pages
client/src/components/	RideCard, ChatBox, SeatMap, FareSplitDisplay
client/src/context/	AuthContext (global auth + JWT), SocketContext (Socket.io)
server/routes/	Express route handlers: auth, rides, messages
server/services/	RideService, AuthService, NotificationService, ChatService, FareSplitter
server/patterns/	NotificationFactory (Factory Pattern), SocketManager (Singleton)
server/repositories/	RideRepository and UserRepository (DB query layer)
server/middleware/	JWT verification middleware
server/sockets/	Socket.io event handlers (chatSocket.js)
server/models/	Mongoose models — User, Ride, RideMember, Message, Notification

13.2 Environment Variables

Create a server/.env file with the following variables:

- MONGODB_URI — MongoDB connection string (e.g. mongodb+srv://...)
- JWT_SECRET — Random secret key, minimum 32 characters
- EMAIL_USER — Gmail address for OTP dispatch
- EMAIL_PASS — 16-character Gmail app password
- PORT — Backend port (default: 5000)

Create a client/.env file with:

- VITE_API_URL — Backend URL (e.g., http://localhost:5000)
- VITE_SOCKET_URL — Socket.io URL (e.g., http://localhost:5000)

13.3 Setup Instructions

1. Clone the repository: git clone https://github.com/[team]/cabshare.git
2. Install backend dependencies: cd server && npm install
3. Install frontend dependencies: cd client && npm install
4. Create server/.env with the variables listed above
5. Start MongoDB connection and sync Mongoose models: node scripts/db-init.js

6. Start backend: npm run dev (runs on port 5000)
7. Start frontend: npm run dev (runs on port 5173)
8. Open: http://localhost:5173

14. Team Members & Contributions

This project is submitted by a group of 5 students. Each member owned a specific module and is responsible for individual commits visible in the GitHub repository.

#	Name	Roll No.	Modules Owned
1	Vani Rudra	2401010490	Team Lead + Backend Architecture — Auth, Ride Service, SOLID design, README & Repo setup
2	Bulbul Agarwalla	2401010131	Frontend Development — All React pages, AuthContext, routing, Class Diagram
3	Anuradha Raghuwanshi	2401010087	Real-Time & Chat — Socket.io, ChatBox component, live updates, Use Case Diagram
4	Apoorva Choudhary	2401010092	Database & API Design — Mongoose models, REST endpoints, ER Diagram
5	Ganga Raghuwanshi	2401010165	Testing & Documentation — Test cases, Sequence Diagram, Report writing